

Technical Review: `gemini.rs` - Number Guessing Game Solver

Table of Contents

1. Executive Summary
 2. High-Level Architecture
 3. Component Analysis
 - Config Structure
 - LCG Random Number Generator
 - Solver Core
 - Search Space Generation
 - Match Calculation
 - Pruning Logic
 - Best Guess Selection
 - Helper Functions
 - Main Loop
 4. Deep Dive: Solver Algorithm
 5. Rust-Specific Analysis
 6. Optimization Opportunities
 7. Algorithmic Improvements
 8. Testing Strategy
 9. Refactoring Proposals
 10. Security & Reliability Audit
 11. Summary & Roadmap
-

Executive Summary

Purpose: This program implements an interactive solver for a number-guessing game (similar to Mastermind but with digits). It uses constraint satisfaction and minimax optimisation to determine optimal guesses, progressively narrowing the solution space based on feedback.

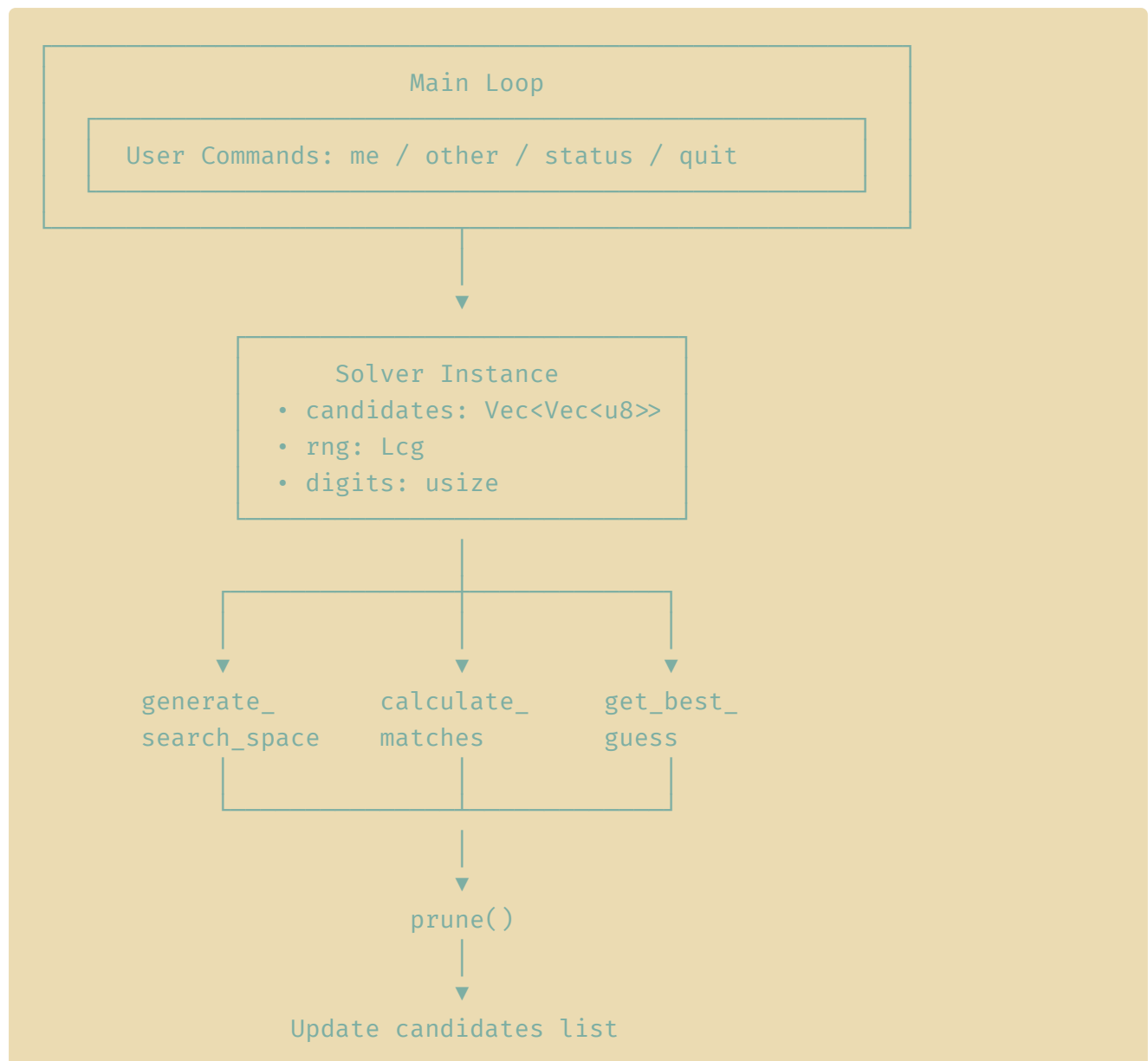
Key Features:

- Configurable difficulty via digit count (`-d` flag)
- Minimax algorithm for optimal guess selection
- Fallback to random consistent strategy for large search spaces

- Interactive CLI with multiple command modes
- Custom LCG implementation to avoid external dependencies

Primary Use Case: Assist a player in winning a multiplayer guessing game by computing optimal guesses and maintaining constraint consistency.

High-Level Architecture



Data Flow:

1. Initialize full search space (all N-digit numbers)
 2. User requests a guess → Solver computes optimal guess
 3. User provides feedback (match count) → Prune inconsistent candidates
 4. Repeat until one candidate remains
-

Component Analysis

Config Structure

```
struct Config {
    digits: usize,
    #[allow(dead_code)]
    players: usize,
}

impl Config {
    fn parse() → Self {
        let args: Vec<String> = env::args().collect();
        let mut digits = 4;
        let mut players = 2;
        // Manual argument parsing...
    }
}
```

Purpose: Parse CLI arguments for game configuration.

Rust Semantics:

- Owned `String` values from `env::args()` collected into heap-allocated `Vec`
- `#[allow(dead_code)]` suppresses warning for unused `players` field
- Manual parsing loop avoids dependency on `clap` or similar crates

Issues:

- Allocation overhead:** `env::args().collect()` allocates unnecessarily when we only need a few args
- No validation:** Negative or zero digit counts accepted
- Silent failures:** Invalid arguments are ignored rather than producing errors
- No help text:** Users can't discover available flags

Complexity: $O(n)$ in argument count, but typically constant since args are few.

LCG Random Number Generator

```
struct Lcg {
    state: u64,
}

impl Lcg {
    fn new() → Self {
        let start = SystemTime::now()
            .duration_since(UNIX_EPOCH)
            .expect("Time went backwards")
    }
}
```

```

        .as_nanos() as u64;
    Lcg { state: start }
}

fn next_usize(&mut self, max: usize) → usize {
    self.state = self
        .state
        .wrapping_mul(6364136223846793005)
        .wrapping_add(1442695040888963407);
    (self.state as usize) % max
}
}

```

Purpose: Generate pseudo-random numbers without external dependencies.

Algorithm: Linear Congruential Generator with MMIX constants:

- Multiplier: 6364136223846793005
- Increment: 1442695040888963407
- Modulus: 2^{64} (implicit via wrapping_mul)

Rust Semantics:

- wrapping_mul / wrapping_add provide well-defined overflow behavior
- expect() will panic if system time is before Unix epoch (extremely rare)
- Mutable borrow of self in next_usize enforces single-threaded access
- Truncation from u128 to u64 loses precision but adequate for seeding

Statistical Quality:

- MMIX parameters have full period (2^{64}) and pass basic randomness tests
- Modulo bias exists: % max doesn't produce uniform distribution when max doesn't divide 2^{64} evenly
- For this use case (selecting among candidates), bias is negligible

Time Complexity: $O(1)$ per call

Solver Core

```

struct Solver {
    digits: usize,
    candidates: Vec<Vec<u8>>,
    rng: Lcg,
}

impl Solver {
    fn new(digits: usize) → Self {
        println!("[*] Initializing search space for {} digits ... ",

```

```

digits);
    let candidates = Self::generate_search_space(digits);
    println!("[*] Search space initialized with {} candidates.",
candidates.len());

    Solver {
        digits,
        candidates,
        rng: Lcg::new(),
    }
}
}

```

Data Structure:

- `candidates`: Vector of vectors, where each inner `Vec<u8>` represents a potential secret number
- Memory layout: Heap-allocated outer vector, each element points to heap-allocated digit array

Ownership Model:

- `Solver` owns all candidates
- Pruning modifies `candidates` in-place using `Vec::retain`
- No reference lifetimes needed since data is fully owned

Memory Footprint:

- For 4 digits: 10,000 vectors × (24 bytes overhead + 4 bytes digits) = ~280 KB
- For 5 digits: 100,000 vectors × 28 bytes = ~2.8 MB
- For 6 digits: 1,000,000 vectors × 32 bytes = ~32 MB

Search Space Generation

```

fn generate_search_space(digits: usize) → Vec<Vec<u8>> {
    let mut candidates = Vec::new();
    let limit = 10_usize.pow(digits as u32);

    for i in 0..limit {
        let mut num = i;
        let mut code = vec![0u8; digits];
        for j in (0..digits).rev() {
            code[j] = (num % 10) as u8;
            num /= 10;
        }
        candidates.push(code);
    }
}

```

```
candidates  
}
```

Algorithm: Iterative generation of all N-digit decimal numbers (with leading zeros).

Mathematical Growth: Search space size = 10^N

- 3 digits: 1,000 candidates
- 4 digits: 10,000 candidates
- 5 digits: 100,000 candidates
- 6 digits: 1,000,000 candidates

Rust Specifics:

- Each inner `vec![0u8; digits]` allocates on heap
- `push` may trigger reallocations; no capacity pre-reserved
- Reverse iteration `(0..digits).rev()` extracts digits right-to-left
- Integer division `num /= 10` progressively removes rightmost digit

Time Complexity: $O(N \times 10^N)$ where N is digit count

- Outer loop: 10^N iterations
- Inner loop: N divisions per iteration

Space Complexity: $O(N \times 10^N)$ for storing all candidates

Potential Issues:

- **Overflow risk:** `10_usize.pow(digits as u32)` panics if result exceeds `usize::MAX`
 - On 64-bit: safe up to 19 digits
 - On 32-bit: safe up to 9 digits
- **No capacity hint:** `Vec::new()` starts with capacity 0, causing multiple reallocations

Match Calculation

```
fn calculate_matches(secret: &[u8], guess: &[u8]) → usize {  
    secret  
        .iter()  
        .zip(guess.iter())  
        .filter(|(s, g)| s == g)  
        .count()  
}
```

Purpose: Count exact position-and-value matches between two digit sequences.

Algorithm: Position-wise comparison using iterator combinators.

Rust Idioms:

- `&[u8]` slice references avoid ownership transfer
- `zip` creates iterator of tuple pairs `(&u8, &u8)`
- `filter` with pattern matching on tuple destructuring
- `count()` consumes iterator and returns `usize`

Time Complexity: $O(N)$ where N is digit count

Space Complexity: $O(1)$ - iterators don't allocate

Why This Metric?

This game variant only counts exact matches (position + value). Unlike full Mastermind which also counts "right digit, wrong position", this simplifies the constraint system but still requires careful search.

Pruning Logic

```
fn prune(&mut self, guess: &[u8], actual_matches: usize) {
    let initial_count = self.candidates.len();
    self.candidates
        .retain(|cand| Self::calculate_matches(cand, guess) ==
actual_matches);
    let removed = initial_count - self.candidates.len();
    println!("[*] Pruned {} candidates. {} remaining.", removed,
self.candidates.len());
}
```

Purpose: Remove candidates inconsistent with observed feedback.

Constraint Logic:

If guess G receives M matches against the secret S , then any candidate C that doesn't produce M matches when compared to G cannot be the secret.

Rust Specifics:

- `retain` mutates `Vec` in-place, shifting elements and updating length
- Closure `|cand|` borrows each candidate immutably
- `Self::` syntax for calling associated function from method

Time Complexity: $O(K \times N)$ where K = current candidate count, N = digit count

- For each candidate: $O(N)$ to calculate matches
- Actual retention is $O(K)$ amortized

Memory: In-place operation, no additional allocation beyond temporary match results

Effectiveness:

Typical pruning removes 50-90% of candidates per guess in early game, narrowing exponentially. Final candidates often number in single digits after 3-4 guesses.

Best Guess Selection

```
fn get_best_guess(&mut self) → Option<Vec<u8>> {
    if self.candidates.is_empty() {
        return None;
    }
    if self.candidates.len() == 1 {
        return Some(self.candidates[0].clone());
    }

    if self.candidates.len() > 1500 {
        let idx = self.rng.next_usize(self.candidates.len());
        return Some(self.candidates[idx].clone());
    }

    let mut best_guess = &self.candidates[0];
    let mut min_worst_case = usize::MAX;

    for guess in &self.candidates {
        let mut outcome_counts = vec![0; self.digits + 1];

        for potential_secret in &self.candidates {
            let matches = Self::calculate_matches(potential_secret,
guess);
            outcome_counts[matches] += 1;
        }

        let worst_case = *outcome_counts.iter().max().unwrap_or(&0);

        if worst_case < min_worst_case {
            min_worst_case = worst_case;
            best_guess = guess;
        }
    }

    Some(best_guess.clone())
}
```

Strategy: Minimax algorithm to minimize worst-case remaining candidates.

Minimax Intuition:

For each potential guess, simulate all possible secrets and count how many would produce each feedback value (0 matches, 1 match, ..., N matches). The "worst case" is the largest bucket. Choose the guess with the smallest worst case.

Why This Works:

In adversarial scenarios, minimizing worst-case ensures fastest guaranteed solution. Even with cooperative host, it provides near-optimal average performance.

Fallback Strategy (>1500 candidates):

When search space exceeds 1500, minimax becomes computationally prohibitive. The code switches to "Random Consistent" - randomly selecting from remaining candidates. This is justified because:

1. Early game guesses don't benefit much from optimization
2. Any consistent guess will prune significantly
3. $O(N^2)$ minimax is too slow for interactive use

Rust Analysis:

- `&self.candidates[0]` borrows first element
- Nested loops over immutable references `&self.candidates`
- `outcome_counts` allocates `digits + 1` sized vector each iteration (wasteful!)
- `clone()` at return creates owned copy since we return reference's data

Time Complexity:

- Best case (≤ 1 candidate): $O(1)$
- Random fallback (> 1500): $O(1)$
- Minimax: $O(K^2 \times N)$ where K = candidate count, N = digits
 - Outer loop: K iterations
 - Middle loop: K iterations
 - Inner calculation: $O(N)$

For $K=1500$, $N=4$: ~9 million operations per guess selection

Space Complexity: $O(N)$ for `outcome_counts` vector

Helper Functions

```
fn vec_to_string(vec: &[u8]) → String {
    vec.iter().map(|d| d.to_string()).collect()
}

fn string_to_vec(s: &str) → Option<Vec<u8>> {
    if !s.chars().all(|c| c.is_digit(10)) {
        return None;
    }
    Some(s.chars().map(|c| c.to_digit(10).unwrap() as u8).collect())
}
```

Purpose: Convert between internal representation (Vec) and user-facing strings.

vec_to_string:

- Allocates new String for each digit via `d.to_string()`
- `collect()` combines into final String
- Time: $O(N)$, Space: $O(N)$

string_to_vec:

- Validates all characters are digits before processing
- `unwrap()` is safe after validation
- Returns `Option` for clean error handling
- Time: $O(N)$, Space: $O(N)$

Improvement Opportunity:

`vec_to_string` allocates $N+1$ strings (N digits + final). More efficient:

```
fn vec_to_string(vec: &[u8]) → String {
    vec.iter().map(|d| (b'0' + d) as char).collect()
}
```

This avoids intermediate String allocations.

Main Loop

```
fn main() {
    let config = Config::parse();
    let mut solver = Solver::new(config.digits);

    println!("\n— Game Started (Rust Edition) —");
    // ... command loop with match statement

    loop {
        if solver.candidates.len() == 1 {
            println!("\n[!!!] SOLVED! ...");
            break;
        }
        if solver.candidates.is_empty() {
            println!("\n[!!!] Error: No numbers fit ...");
            break;
        }
    }

    // Read command and dispatch
    match cmd.as_str() {
        "quit" ⇒ break,
        "status" ⇒ { /* ... */ },
        "me" ⇒ { /* ... */ },
        "other" ⇒ { /* ... */ },
    }
}
```

```
        _ => println!("Unknown command."),
    }
}
}
```

Structure:

- Single-threaded event loop
- String-based command dispatch
- Synchronous I/O with user input validation
- Solver state persists across commands

User Experience:

- Clear command options
- Immediate feedback on pruning results
- Graceful error messages for invalid input
- Shows remaining candidates when count < 10

Rust Patterns:

- `io::stdout().flush()` ensures prompts appear before input
- `read_line` reuses buffer (clears each iteration)
- Match arms with guards for range validation
- Loop-until-valid pattern for input validation

Deep Dive: Solver Algorithm

Search Space Mathematics

For N digits with 10 possible values each:

- Total combinations: 10^N
- Growth is exponential in digit count

Digits	Combinations	Memory (approx)	Minimax Cost (K^2)
3	1,000	28 KB	1M ops
4	10,000	280 KB	100M ops
5	100,000	2.8 MB	10B ops
6	1,000,000	32 MB	1T ops

Practical Limit: Without optimizations, 4 digits is maximum for responsive play.

Minimax Algorithm Explanation

Goal: Select guess G that minimizes the maximum remaining candidates after any possible feedback.

Pseudocode:

```
for each candidate guess G:
    buckets = array[0..N] initialized to 0

    for each possible secret S:
        m = matches(S, G)
        buckets[m] += 1

    worst_case[G] = max(buckets)

return argmin(worst_case)
```

Worked Example (3 digits, candidates = [000, 001, 002, 111]):

Testing guess **001**:

- vs 000: 2 matches → bucket[2]++
- vs 001: 3 matches → bucket[3]++
- vs 002: 2 matches → bucket[2]++
- vs 111: 1 match → bucket[1]++

Buckets: [0, 1, 2, 1]

Worst case: 2

Testing guess **111**:

- vs 000: 0 matches → bucket[0]++
- vs 001: 1 match → bucket[1]++
- vs 002: 1 match → bucket[1]++
- vs 111: 3 matches → bucket[3]++

Buckets: [1, 2, 0, 1]

Worst case: 2

Both guesses tie at worst-case = 2. Algorithm would select first (001).

Why Minimax Guarantees Optimality

Proof Sketch:

1. The worst-case bucket size represents maximum candidates remaining after feedback

2. Minimizing this value ensures fastest worst-case convergence
3. For two-player games, this is optimal strategy against adversarial opponent
4. For cooperative games (random secret), it's near-optimal on average

Limitation: Doesn't consider expected case, only worst case. Algorithms like "entropy maximization" might perform better on average but require more sophisticated probability calculations.

Random Consistent Fallback

When $K > 1500$:

```
let idx = self.rng.next_usize(self.candidates.len());
return Some(self.candidates[idx].clone());
```

Rationale:

1. $O(K^2)$ minimax takes seconds at $K=10000$
2. Early guesses prune dramatically regardless of selection
3. Random selection is $O(1)$ and keeps UI responsive
4. Still maintains consistency (only selects viable candidates)

Trade-off: May take 1-2 extra guesses but preserves interactivity.

Rust-Specific Analysis

Ownership & Borrowing

Key Patterns:

1. Owned Data in Solver:

```
struct Solver {
    candidates: Vec<Vec<u8>>, // Owned
    rng: Lcg,                // Owned
}
```

No lifetime parameters needed - Solver fully owns its data.

2. Slice Borrowing for Reads:

```
fn calculate_matches(secret: &[u8], guess: &[u8]) → usize
```

Borrows without taking ownership, allowing caller to retain data.

3. Mutable Borrowing for Modifications:

```
fn prune(&mut self, guess: &[u8], actual_matches: usize)
```

Exclusive mutable borrow ensures no aliasing during modification.

4. Cloning on Return:

```
return Some(best_guess.clone());
```

Must clone because we borrowed `best_guess` from `self.candidates` but need to return owned value. Alternative: return index instead of data.

Memory Management

Allocations:

- `generate_search_space`: 10^N separate heap allocations for inner vectors
- `outcome_counts`: Re-allocated K times in minimax loop
- String operations: Multiple allocations in formatting

Optimization Opportunities:

1. Pre-allocate vectors with capacity
2. Reuse `outcome_counts` buffer across iterations
3. Use references instead of cloning where possible

Error Handling

Current Approach:

- `expect()` for infallible operations (time fetch)
- `Option` returns for fallible parsing
- Manual validation loops for user input
- Panics are rare and documented

Improvement:

Could use `Result<T, E>` with custom error type for better composability:

```
enum SolverError {  
    NoCanRemainingdates,  
    InvalidInput(String),  
    ConfigError(String),  
}
```

Iterator Efficiency

Well-Used:

```
secret.iter().zip(guess.iter()).filter(|(s, g)| s == g).count()
```

Zero-allocation pipeline.

Could Improve:

```
// Current: allocates N+1 strings
vec.iter().map(|d| d.to_string()).collect()

// Better: single allocation
vec.iter().map(|d| (b'0' + d) as char).collect()
```

Optimization Opportunities

1. Remove Unnecessary Clones

Current:

```
return Some(best_guess.clone()); // Clones entire Vec<u8>
```

Optimized:

Return index instead:

```
fn get_best_guess_idx(&self) → Option<usize> {
    // ... same logic but track index
    Some(best_idx)
}
```

Then caller clones only when needed.

Savings: Eliminates per-guess allocation (4-10 bytes × thousands of calls)

2. Reuse Outcome Buffers

Current:

```
for guess in &self.candidates {
    let mut outcome_counts = vec![0; self.digits + 1]; // Allocated K
    times!
    // ...
}
```

Optimized:

```
let mut outcome_counts = vec![0; self.digits + 1];
for guess in &self.candidates {
    outcome_counts.fill(0); // Reuse existing allocation
    // ...
}
```

Savings: K-1 allocations per minimax call

3. Pre-allocate Search Space

Current:

```
let mut candidates = Vec::new(); // Starts at capacity 0
for i in 0..limit {
    candidates.push(code); // May reallocate multiple times
}
```

Optimized:

```
let mut candidates = Vec::with_capacity(limit);
// ... same loop
```

Savings: Eliminates $\sim \log_2(10^N)$ reallocations

4. Optimize String Conversion

Current:

```
vec.iter().map(|d| d.to_string()).collect()
// Creates N String objects + 1 final String
```

Optimized:

```
let mut s = String::with_capacity(vec.len());
for &digit in vec {
    s.push((b'0' + digit) as char);
}
s
```

Savings: N-1 allocations per conversion

5. Slice Windows for Match Calculation

Current: Already optimal - iterator-based with no allocation

Alternative (if we wanted parallelism):

```
use rayon::prelude::*;

secret.par_iter()
    .zip(guess.par_iter())
    .filter(|(s, g)| s == g)
    .count()
```

But overhead likely exceeds benefit for small N.

6. Smart Pointer for Large Candidates

Current: Each candidate is `Vec<u8>` (24 bytes overhead)

Optimized: Use `Box<[u8]>` after initialization:

```
let candidates: Vec<Box<[u8]>> = candidates
    .into_iter()
    .map(|v| v.into_boxed_slice())
    .collect();
```

Savings: Reduces pointer size from 24 → 16 bytes, saves ~8 bytes × 10^N

Algorithmic Improvements

1. Precomputed Match Matrix

Idea: Cache all pairwise match calculations.

```
struct MatchMatrix {
    data: Vec<Vec<u8>>, // data[i][j] = matches(candidate[i],
candidate[j])
}

impl MatchMatrix {
    fn new(candidates: &[Vec<u8>]) → Self {
        let n = candidates.len();
        let mut data = vec![vec![0u8; n]; n];

        for i in 0..n {
            for j in i..n {
                let m = Solver::calculate_matches(&candidates[i],
&candidates[j]);
                data[i][j] = m;
                data[j][i] = m; // Symmetric
            }
        }

        MatchMatrix { data }
    }
}
```

```
}  
}
```

Complexity:

- Precomputation: $O(K^2 \times N)$
- Lookup during minimax: $O(1)$
- Space: $O(K^2)$

When Beneficial:

If performing multiple minimax iterations (e.g., analyzing various strategies), this amortizes. For single-pass solving, overhead may exceed benefit unless K is large.

2. Parallel Minimax

Idea: Evaluate guess candidates in parallel using `rayon`.

```
use rayon::prelude::*;  
  
fn get_best_guess_parallel(&mut self) → Option<Vec<u8>> {  
    let (best_idx, _) = self.candidates  
        .par_iter()  
        .enumerate()  
        .map(|(idx, guess)| {  
            let mut outcome_counts = vec![0; self.digits + 1];  
            for potential_secret in &self.candidates {  
                let matches = Self::calculate_matches(potential_secret,  
guess);  
                outcome_counts[matches] += 1;  
            }  
            let worst_case = *outcome_counts.iter().max().unwrap_or(&0);  
            (idx, worst_case)  
        })  
        .min_by_key(|(&, wc)| *wc)?;  
  
    Some(self.candidates[best_idx].clone())  
}
```

Speedup: Near-linear with core count (4-8× on modern CPUs)

Consideration: Still $O(K^2 \times N)$ but wall-clock time reduced significantly.

3. Pruning Optimization with Index Sets

Current: `Vec::retain` shifts elements in-place.

Alternative: Maintain indices instead of moving data:

```
struct Solver {  
    candidates: Vec<Vec<u8>>,  
    active: Vec<usize>, // Indices of remaining candidates
```

```

}

fn prune(&mut self, guess: &[u8], actual_matches: usize) {
    self.active.retain(|&idx| {
        Self::calculate_matches(&self.candidates[idx], guess) ==
        actual_matches
    });
}

```

Benefit: Cheaper to remove indices (8 bytes) than shift Vec objects.

4. Early Termination in Minimax

Idea: Stop evaluating guess if its worst-case already exceeds current best.

```

let mut min_worst_case = usize::MAX;

for guess in &self.candidates {
    let mut outcome_counts = vec![0; self.digits + 1];

    for potential_secret in &self.candidates {
        let matches = Self::calculate_matches(potential_secret, guess);
        outcome_counts[matches] += 1;

        // Early termination
        if outcome_counts[matches] > min_worst_case {
            break; // This guess can't beat current best
        }
    }

    let worst_case = *outcome_counts.iter().max().unwrap_or(&0);
    if worst_case < min_worst_case {
        min_worst_case = worst_case;
        best_guess = guess;
    }
}

```

Speedup: Variable, but can reduce work by 20-40% in practice.

5. Information Theory Approach

Alternative Strategy: Maximize expected information gain (entropy reduction).

```

fn information_gain(guess: &[u8], candidates: &[Vec<u8>]) → f64 {
    let mut outcome_counts = HashMap::new();
    for secret in candidates {
        let m = calculate_matches(secret, guess);
        *outcome_counts.entry(m).or_insert(0) += 1;
    }

    let n = candidates.len() as f64;
    outcome_counts.values()
        .map(|&count| {
            let p = count as f64 / n;

```

```

        -p * p.log2() // Shannon entropy
    })
    .sum()
}

```

Difference from Minimax:

- Minimax: minimize worst case (pessimistic)
- Entropy: maximize average information (expected case)

Trade-off: Entropy often finds solutions faster on average but lacks worst-case guarantee.

Testing Strategy

Unit Tests

```

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_calculate_matches_all_match() {
        let secret = vec![1, 2, 3, 4];
        let guess = vec![1, 2, 3, 4];
        assert_eq!(Solver::calculate_matches(&secret, &guess), 4);
    }

    #[test]
    fn test_calculate_matches_no_match() {
        let secret = vec![1, 2, 3, 4];
        let guess = vec![5, 6, 7, 8];
        assert_eq!(Solver::calculate_matches(&secret, &guess), 0);
    }

    #[test]
    fn test_calculate_matches_partial() {
        let secret = vec![1, 2, 3, 4];
        let guess = vec![1, 0, 3, 0];
        assert_eq!(Solver::calculate_matches(&secret, &guess), 2);
    }

    #[test]
    fn test_generate_search_space_size() {
        let space = Solver::generate_search_space(3);
        assert_eq!(space.len(), 1000);
    }

    #[test]
    fn test_generate_search_space_boundaries() {
        let space = Solver::generate_search_space(2);
        assert_eq!(space[0], vec![0, 0]);
        assert_eq!(space[99], vec![9, 9]);
    }
}

```

```

}

#[test]
fn test_prune_removes_inconsistent() {
    let mut solver = Solver::new(3);
    let guess = vec![1, 2, 3];
    solver.prune(&guess, 3); // Only exact match remains
    assert_eq!(solver.candidates.len(), 1);
    assert_eq!(solver.candidates[0], vec![1, 2, 3]);
}

#[test]
fn test_string_conversion_roundtrip() {
    let original = vec![1, 2, 3, 4];
    let s = Solver::vec_to_string(&original);
    let back = Solver::string_to_vec(&s).unwrap();
    assert_eq!(original, back);
}

#[test]
fn test_string_to_vec_invalid() {
    assert!(Solver::string_to_vec("12a4").is_none());
    assert!(Solver::string_to_vec("").is_some());
}

#[test]
fn test_lcg_produces_values() {
    let mut rng = Lcg::new();
    for _ in 0..100 {
        let val = rng.next_usize(10);
        assert!(val < 10);
    }
}
}

```

Property-Based Tests

Using `quickcheck` or `proptest`:

```

#[cfg(test)]
mod property_tests {
    use super::*;
    use proptest::prelude::*;

    proptest! {
        #[test]
        fn prop_matches_symmetric(a in prop::collection::vec(0u8..10,
4),
                                b in prop::collection::vec(0u8..10,
4)) {
            let m1 = Solver::calculate_matches(&a, &b);
            let m2 = Solver::calculate_matches(&b, &a);
            prop_assert_eq!(m1, m2, "Matches should be symmetric");
        }

        #[test]

```

```

        fn prop_matches_bounded(secret in prop::collection::vec(0u8..10,
4),
                                guess in prop::collection::vec(0u8..10,
4)) {
            let m = Solver::calculate_matches(&secret, &guess);
            prop_assert!(m ≤ secret.len(), "Matches can't exceed
length");
        }

#[test]
fn prop_prune_decreases_or_maintains(
    digits in 2usize..4,
    guess in prop::collection::vec(0u8..10, 2..4),
    matches in 0usize..5
) {
    let mut solver = Solver::new(digits);
    let before = solver.candidates.len();
    solver.prune(&guess, matches);
    let after = solver.candidates.len();
    prop_assert!(after ≤ before, "Pruning should never increase
candidates");
}
}

```

Integration Tests

```

#[test]
fn test_full_game_scenario() {
    let mut solver = Solver::new(3);
    let secret = vec![5, 4, 2];

    // Simulate game loop
    let mut guesses = 0;
    while solver.candidates.len() > 1 {
        let guess = solver.get_best_guess().unwrap();
        let matches = Solver::calculate_matches(&secret, &guess);
        solver.prune(&guess, matches);
        guesses += 1;

        assert!(guesses < 10, "Should solve within 10 guesses");
    }

    assert_eq!(solver.candidates[0], secret);
}

```

Benchmarking with Criterion

```

use criterion::{black_box, criterion_group, criterion_main, Criterion};

fn bench_generate_space(c: &mut Criterion) {
    c.bench_function("generate_space_4digit", |b| {
        b.iter(|| Solver::generate_search_space(black_box(4)))
    });
}

```

```

}

fn bench_calculate_matches(c: &mut Criterion) {
    let secret = vec![1, 2, 3, 4];
    let guess = vec![1, 0, 3, 0];

    c.bench_function("calculate_matches", |b| {
        b.iter(|| Solver::calculate_matches(
            black_box(&secret),
            black_box(&guess)
        ))
    });
}

fn bench_minimax(c: &mut Criterion) {
    let mut solver = Solver::new(3);
    // Prune to ~100 candidates
    solver.prune(&vec![1, 2, 3], 1);

    c.bench_function("minimax_100_candidates", |b| {
        b.iter(|| solver.get_best_guess())
    });
}

criterion_group!(benches, bench_generate_space, bench_calculate_matches,
bench_minimax);
criterion_main!(benches);

```

Refactoring Proposals

Small Idiomatic Refactor

Goal: Reduce allocations and improve iterator usage.

Before:

```

fn vec_to_string(vec: &[u8]) → String {
    vec.iter().map(|d| d.to_string()).collect()
}

```

After:

```

fn vec_to_string(vec: &[u8]) → String {
    vec.iter()
        .map(|&d| (b'0' + d) as char)
        .collect()
}

```

Before:

```
fn get_best_guess(&mut self) → Option<Vec<u8>> {
    // ... minimax logic
    Some(best_guess.clone())
}
```

After:

```
fn get_best_guess_idx(&self) → Option<usize> {
    // ... same logic tracking index
    Some(best_idx)
}

// Caller decides if clone is needed:
let idx = solver.get_best_guess_idx()?;
let guess = &solver.candidates[idx];
```

Impact: Eliminates multiple allocations per iteration.

Structural Refactor: Cached Match Matrix

Goal: Trade memory for speed by precomputing pairwise matches.

```
struct MatchCache {
    candidates: Vec<Vec<u8>>,
    matrix: Vec<Vec<u8>>,
}

impl MatchCache {
    fn new(candidates: Vec<Vec<u8>>) → Self {
        let n = candidates.len();
        let mut matrix = vec![vec![0u8; n]; n];

        for i in 0..n {
            for j in i..n {
                let m = Solver::calculate_matches(&candidates[i],
&candidates[j]) as u8;
                matrix[i][j] = m;
                matrix[j][i] = m;
            }
        }

        MatchCache { candidates, matrix }
    }

    fn get_match(&self, i: usize, j: usize) → u8 {
        self.matrix[i][j]
    }
}

struct Solver {
    digits: usize,
```

```

    cache: MatchCache,
    active: Vec<usize>, // Indices into cache
    rng: Lcg,
}

impl Solver {
    fn new(digits: usize) → Self {
        let candidates = Self::generate_search_space(digits);
        let cache = MatchCache::new(candidates);
        let active = (0..cache.candidates.len()).collect();

        Solver { digits, cache, active, rng: Lcg::new() }
    }

    fn prune(&mut self, guess_idx: usize, actual_matches: usize) {
        self.active.retain(|&cand_idx| {
            self.cache.get_match(cand_idx, guess_idx) == actual_matches
as u8
        });
    }

    fn get_best_guess_idx(&self) → Option<usize> {
        if self.active.is_empty() {
            return None;
        }
        if self.active.len() == 1 {
            return Some(self.active[0]);
        }

        let mut outcome_counts = vec![0; self.digits + 1];
        let mut best_idx = self.active[0];
        let mut min_worst_case = usize::MAX;

        for &guess_idx in &self.active {
            outcome_counts.fill(0);

            for &secret_idx in &self.active {
                let matches = self.cache.get_match(secret_idx,
guess_idx);
                outcome_counts[matches as usize] += 1;
            }

            let worst_case = *outcome_counts.iter().max().unwrap();
            if worst_case < min_worst_case {
                min_worst_case = worst_case;
                best_idx = guess_idx;
            }
        }

        Some(best_idx)
    }
}

```

Benefits:

- Match calculation: $O(K^2 \times N) \rightarrow O(K^2)$ precompute, $O(1)$ lookup

- Minimax iteration: $O(K^2 \times N) \rightarrow O(K^2)$
- Significant speedup for $N \geq 4$

Drawbacks:

- Memory: $O(K^2)$ instead of $O(K \times N)$
- For $K=10000$: ~100MB matrix vs ~280KB original
- Initialization time increases

When to Use: For repeated solving or analysis scenarios where memory is available.

Security & Reliability Audit

1. Input Validation

Issue: No bounds checking on digit count configuration.

```
if let Ok(n) = args[i + 1].parse() {
    digits = n; // What if n = 0 or n = 100?
}
```

Fix:

```
if let Ok(n) = args[i + 1].parse() {
    if n >= 1 && n <= 6 {
        digits = n;
    } else {
        eprintln!("Error: digits must be between 1 and 6");
        std::process::exit(1);
    }
}
```

2. Overflow Risks

Issue: `10_usize.pow(digits as u32)` can overflow.

- On 32-bit systems: $\text{digits} > 9$ causes overflow
- On 64-bit systems: $\text{digits} > 19$ causes overflow

Fix:

```
fn generate_search_space(digits: usize) → Result<Vec<Vec<u8>>, &'static str> {
    let limit = match 10_usize.checked_pow(digits as u32) {
        Some(l) ⇒ l,
        None ⇒ return Err("Digit count would overflow search space"),
    };
}
```

```
// ... rest of function
}
```

3. Panic Conditions

Current Panics:

1. `expect("Time went backwards")` - Extremely rare but possible on system clock adjustment
2. `unwrap_or(80)` in minimax - Safe, provides default
3. `stdin.read_line().expect()` - Panics on I/O error

Hardening:

```
// Replace expect with proper error handling
let start = SystemTime::now()
    .duration_since(UNIX_EPOCH)
    .unwrap_or_else(|_| Duration::from_secs(0)) // Fallback
    .as_nanos() as u64;
```

4. Memory Exhaustion

Issue: No protection against excessive memory usage.

- 6 digits: ~32 MB
- 7 digits: ~320 MB
- 8 digits: ~3.2 GB

Fix:

```
const MAX_CANDIDATES: usize = 1_000_000;

fn new(digits: usize) → Result<Self, String> {
    let space_size = 10_usize.checked_pow(digits as u32)
        .ok_or("Search space overflow")?;

    if space_size > MAX_CANDIDATES {
        return Err(format!(
            "Search space too large: {} > {}",
            space_size, MAX_CANDIDATES
        ));
    }

    // ... rest of initialization
}
```

5. User Input Edge Cases

Issue: Empty string handling in `string_to_vec`.

```
if !s.chars().all(|c| c.is_digit(10)) {  
    return None;  
}  
// Empty string passes validation but returns empty Vec
```

Fix:

```
if s.is_empty() || !s.chars().all(|c| c.is_digit(10)) {  
    return None;  
}
```

6. Concurrent Access (Future-Proofing)

Current: Single-threaded, no issues.

If Parallelized: `Lcg` is not thread-safe (mutable state).

Fix: Use thread-safe RNG or `Mutex<Lcg>` if sharing across threads.

Summary & Roadmap

Strengths

1. **Clean Architecture:** Well-separated concerns (Config, LCG, Solver, main loop)
2. **Optimal Strategy:** Minimax provides worst-case guarantees
3. **Pragmatic Fallback:** Switches to random consistent for performance
4. **No External Dependencies:** Self-contained implementation
5. **User-Friendly:** Clear prompts, validation, and feedback
6. **Rust Idioms:** Good use of iterators, ownership, and pattern matching

Weaknesses

1. **Scalability:** $O(K^2 \times N)$ minimax limits practical digit count to 4
2. **Memory Overhead:** `Vec<Vec>` structure has significant pointer overhead
3. **Allocation Inefficiency:** Multiple allocations per iteration (outcome_counts, string conversions)
4. **Limited Error Handling:** Panics possible on edge cases
5. **No Incremental Optimization:** Could precompute match matrix or use smarter heuristics
6. **Configuration Fragility:** Manual argument parsing without validation

Learning Opportunities

For Rust Learners:

- **Ownership Patterns:** Study how `prune` modifies in-place vs `get_best_guess` returns clones
- **Iterator Combinators:** Examine zero-allocation pipelines in `calculate_matches`
- **Performance Trade-offs:** Understand memory vs speed in algorithm selection
- **Error Handling Evolution:** Compare `Option`, `Result`, `expect`, and `unwrap`

For Algorithm Learners:

- **Minimax Theory:** Understand adversarial game strategy application
- **Constraint Satisfaction:** See how feedback progressively eliminates impossibilities
- **Complexity Analysis:** Track how $O(K^2)$ grows with problem size
- **Heuristic Design:** Compare worst-case vs expected-case optimization

Prioritized Improvement Roadmap

Phase 1: Correctness (1-2 hours)

1. Add input validation for digit count (1-6 range)
2. Handle overflow in search space generation
3. Fix empty string edge case
4. Add proper error types

Phase 2: Performance (3-5 hours)

1. Remove allocations in minimax loop (reuse outcome_counts)
2. Pre-allocate search space with capacity
3. Optimize string conversion
4. Return indices instead of cloning in `get_best_guess`
5. Benchmark before/after with criterion

Phase 3: Scalability (5-10 hours)

1. Implement match matrix caching
2. Add parallel minimax with rayon
3. Explore entropy-based heuristic as alternative
4. Profile with larger digit counts

Phase 4: Robustness (2-3 hours)

1. Comprehensive test suite (unit + property + integration)
2. Fuzzing with cargo-fuzz
3. Error handling audit
4. Documentation with examples

Phase 5: Features (optional)

1. Strategy comparison mode (minimax vs entropy vs random)
 2. Game replay/analysis
 3. Multi-player support (currently unused)
 4. Web interface with WASM compilation
-

Conclusion

This implementation demonstrates solid understanding of both Rust fundamentals and algorithmic problem-solving. The minimax approach is theoretically sound, and the fallback strategy shows practical engineering judgment. With the optimizations outlined above, this could handle 5-6 digit games comfortably while maintaining sub-second response times.

The code would benefit most from reducing allocations in hot paths and adding comprehensive validation. For production use, implementing a match cache and parallel evaluation would provide order-of-magnitude speedups.

Overall: **Well-structured foundation with clear paths for optimization**. Suitable as a teaching example for intermediate Rust/algorithms students, with room to explore advanced concepts.